

Q1. Explain Von-Neumann (8086) Architecture in detail. Explain all its functional units.

The 8086 microprocessor architecture is based on the **Von Neumann architecture model** (though implemented with modified Harvard architectural elements internally via the Bus Interface Unit and Execution Unit). It features a 16-bit ALU, 16-bit internal registers, and a 20-bit address bus capable of addressing 1 MB of memory.

Logically, the 8086 CPU is divided into two independent functional units:

1. Bus Interface Unit (BIU):

- **Role:** Handles all transfers of data and addresses on the external buses for the execution unit. It fetches instructions, reads data from memory/ports, and writes data to memory/ports.
- **Instruction Queue:** A 6-byte prefetch queue. While the EU is executing instructions, the BIU fetches up to 6 subsequent instruction bytes from memory, implementing *pipelining* to speed up execution.
- **Segment Registers:** CS (Code Segment), DS (Data Segment), SS (Stack Segment), and ES (Extra Segment) to implement memory segmentation ($20\text{-bit physical address} = \text{Segment} \times 16 + \text{Offset}$).
- **Instruction Pointer (IP):** Holds the 16-bit offset of the next instruction to be fetched.

2. Execution Unit (EU):

- **Role:** Decodes and executes instructions received from the BIU queue.
- **Arithmetic Logic Unit (ALU):** 16-bit ALU performs arithmetic (ADD, SUB, MUL, DIV) and logical (AND, OR, XOR) operations.
- **General Purpose Registers:** AX, BX, CX, DX (can be accessed as 16-bit or split into 8-bit high/low registers like AH, AL).
- **Pointer and Index Registers:** SP (Stack Pointer), BP (Base Pointer), SI (Source Index), DI (Destination Index).
- **Flag Register:** A 16-bit register containing 9 active flags (6 status flags: CF, PF, AF, ZF, SF, OF; 3 control flags: TF, IF, DF).

Q2. Explain basic computer architecture (IAS). How further, upcoming architectures were evolved further.

IAS (Institute for Advanced Study) Computer Architecture: Designed by John von Neumann and his team in 1946 at Princeton, it established the foundational stored-program computer model.

- **Main Components:**

1. **Memory:** Stores both data and instructions (Total 1098 words, 40 bits per word).
2. **Arithmetic Logic Unit (ALU):** Operates on binary data.
3. **Program Control Unit:** Interprets instructions in memory and causes them to be executed.
4. **Input/Output Equipment:** Operated by the control unit.

- **Registers in IAS:** Accumulator (AC), Multiplier-Quotient (MQ), Program Counter (PC), Memory Address Register (MAR), Memory Buffer Register (MBR), Instruction Register (IR).

Evolution of Upcoming Architectures:

- **Von Neumann Bottleneck Mitigation:** Early IAS suffered from sequential instruction/data access bottlenecks. Evolution led to *Harvard Architecture* (separate instruction and data buses/caches).
- **Pipelining & Superscalar Execution:** Moved from single-cycle execution to multi-stage pipelining (fetching, decoding, executing simultaneously) and superscalar execution (multiple instructions per clock cycle).
- **Multicore & Parallel Processing:** Evolution from single-core processors to multi-core chips, GPU acceleration, and distributed computing architectures to handle massive parallelism.

Q3. Explain various modes of retrieval of instructions and data from the memory.

Memory retrieval modes refer to how the CPU accesses instructions and data from the memory hierarchy:

1. **Sequential Access:** Instructions and data are accessed in consecutive memory locations (e.g., using Program Counter increments: $PC \leftarrow PC + 1$).
2. **Random Access (Direct Access):** Any memory location can be accessed directly in constant time using its physical address via address buses and decoders.
3. **Associative Access (Content Addressable Memory - CAM):** Data is retrieved based on its content/value rather than its explicit address location (commonly used in CPU Translation Lookaside Buffers - TLBs and caches).
4. **Cache-Based Retrieval (Hierarchical Access):** Leverages temporal and spatial locality. When CPU requests data, hardware checks L1/L2/L3 caches before accessing slower main memory (DRAM).
5. **Direct Memory Access (DMA):** Peripheral devices transfer data directly to/from main memory without constant CPU intervention, speeding up bulk data retrieval.

Q4. What are the different addressing modes (8086)? Explain all in detail.

Addressing modes specify how the operand of an instruction is accessed. The 8086 microprocessor supports several flexible addressing modes categorized into data-addressing and control-transfer modes:

- **1. Immediate Addressing:** The operand is explicitly specified in the instruction.
Example: `MOV AX, 2500H` (Data 2500H is moved into AX).
- **2. Register Addressing:** The operand is inside a CPU register.
Example: `MOV AX, BX` (Contents of BX moved to AX).
- **3. Direct Addressing:** The 16-bit memory offset of the operand is given directly in the instruction.
Example: `MOV AL, [1000H]`.
- **4. Register Indirect Addressing:** The memory offset is held in a base or index register (BX, BP, SI, DI).
Example: `MOV AX, [BX]`.
- **5. Based Addressing:** The effective address (EA) is formed by adding an 8-bit or 16-bit displacement to a base register (BX or BP).
Formula: $EA = [BX/BP] + displacement$.
- **6. Indexed Addressing:** The EA is formed by adding a displacement to an index register (SI or DI).
Formula: $EA = [SI/DI] + displacement$.
- **7. Based-Indexed Addressing:** The EA is formed by combining a base register and an index register.
Formula: $EA = [BX/BP] + [SI/DI]$.
- **8. Based-Indexed with Displacement:** Combines base register, index register, and displacement.
Formula: $EA = [BX/BP] + [SI/DI] + displacement$.

Q5. Explain below instructions with its addressing modes:

- **5.1 MOV DX, 2500H**

Addressing Mode: **Immediate Addressing Mode.**

Explanation: The 16-bit immediate hex value **2500H** is directly loaded into the destination register **DX**.

- **5.2 MOV AL, [1000H]**

Addressing Mode: **Direct Addressing Mode.**

Explanation: The 16-bit memory offset **1000H** (combined with DS) specifies the memory location whose 8-bit content is moved into register **AL**.

- **5.3 MOV SI, [1010H]** (Note: typo 'S1' corrected to 'SI')

Addressing Mode: **Direct Addressing Mode (Memory-to-Register).**

Explanation: The 16-bit word data stored at memory offset **1010H** in the data segment is loaded into the source index register **SI**.

- **5.4 MOV [BX], CL**

Addressing Mode: **Register Indirect Addressing Mode.**

Explanation: The memory address offset is pointed to by register **BX**. The 8-bit content of register **CL** is stored into that memory location.

- **5.5 MOV AX, [SI+10H]**

Addressing Mode: **Indexed Addressing Mode.**

Explanation: The effective address is calculated by adding the displacement **10H** to the index register **SI**. The 16-bit word at that memory address is loaded into **AX**.

- **5.6 MOV [BP+DI], CX** (Note: formatted notation **[BP+DI] CX**)

Addressing Mode: **Based-Indexed Addressing Mode.**

Explanation: The effective address is computed by adding base register **BP** and index register **DI** (using Stack Segment SS by default). The 16-bit content of **CX** is stored at this computed memory address.

- **5.7 MOV BYTE PTR [DI], 25H**

Addressing Mode: **Register Indirect Addressing Mode with Explicit Size Override.**

Explanation: The **BYTE PTR** directive explicitly informs the assembler that an 8-bit immediate value (**25H**) is to be moved into the memory location pointed to by index register **DI**, overriding default word size.

Q6. Explain bus structure and its types in detail.

A bus is a shared communication pathway that connects multiple subsystems within a computer system (CPU, Memory, I/O devices). It consists of multiple parallel lines.

Types of System Buses:

1. **Address Bus:** Unidirectional bus that carries memory addresses from the CPU to memory or I/O controllers. The width of the address bus determines the maximum addressable memory capacity (N -bit bus addresses 2^N locations).
2. **Data Bus:** Bidirectional bus that transfers actual data/instructions between CPU, memory, and peripherals. Its width (e.g., 8-bit, 16-bit, 32-bit, 64-bit) determines how much data can be transferred in a single clock cycle.
3. **Control Bus:** Bidirectional/mixed control lines that transmit command and timing signals (e.g., Memory Read, Memory Write, I/O Read, I/O Write, Interrupt Request, Bus Grant) to coordinate system operations.

Bus Structures: Single-bus architecture (simple, shared bottleneck), Multi-bus architecture (dedicated memory bus, I/O bus, high-speed peripheral interconnects like PCI Express, USB for improved concurrency).

Q7. Multiply the following using Booth's algorithm: (Multiplicand = +11, Multiplier = -6) and (Multiplicand = 6, Multiplier = 4)

Part (a): Multiplicand = +11 (01011), Multiplier = -6 (11010)

Bit length: 5 bits. $M = 01011$, $-M = 10101$, $Q = 11010$, $Q_{-1} = 0$.

Step	$Q_0 Q_{-1}$	Action	Accumulator (A)	Multiplier (Q)	Q_{-1}
Initial	-	Load initial values	00000	11010	0
1	0 0	Shift Right (Arithmetic)	00000	01105 (01101)	0
2	1 0	$A = A - M$	01011	11010	0
3	...	Complete Booth's Trace	Result in 2's complement = 1110011110 = -66		

Final Product: $+11 \times -6 = -66$ (1111100110 in 2's complement).

Part (b): Multiplicand = 6 (00110), Multiplier = 4 (00100)

$M = 00110$, $-M = 11010$, $Q = 00100$, $Q_{-1} = 0$.

Step	$Q_0 Q_{-1}$	Action	A	Q	Q_{-1}
Initial	-	Setup	00000	00100	0
1	0 0	Shift Right	00000	00010	0
2	0 0	Shift Right	00000	00001	0
3	1 0	$A = A - M$	11010	00001	0
4	...	Shift Right & Add M	Final A & Q = 00000 00011000 = +24		

Final Product: $6 \times 4 = +24$ (00000011000).

Q8. Represent below numbers in 32-bit single precision and 64-bit double precision format: $(-65.23)_{10}$, $(124.16)_{10}$

Part (a): -65.23_{10}

- **Sign bit (S):** Negative number, so $S = 1$.
- **Binary Conversion of 65.23 :**
 - Integer part $65_{10} = 1000001_2$
 - Fractional part $0.23_{10} \approx 0.00111010..._2$
 - Combined: $1000001.0011101011..._2$
- **Normalized Scientific Notation:** $1.00000100111... \times 2^6$
- **Single Precision (32-bit IEEE 754):**
 - Bias = 127. Exponent = $6 + 127 = 133 = 10000101_2$.
 - Mantissa = 00000100111011100001010 .
 - **IEEE 32-bit:** $1\ 10000101\ 00000100111011100001010$
- **Double Precision (64-bit IEEE 754):**
 - Bias = 1023. Exponent = $6 + 1023 = 1029 = 10000000101_2$.
 - Mantissa = 52 bits corresponding to fractional bits of normalized form.

Part (b): 124.16_{10}

- **Sign bit (S):** Positive number, so $S = 0$.
- **Binary Conversion:** $124_{10} = 1111100_2$, $0.16_{10} \approx 0.0010100011..._2$. Normalized = $1.1111000010100011... \times 2^6$.
- **Single Precision (32-bit):**
 - $S = 0$, Exponent = $133\ (10000101)$, Mantissa = 11110000101000111101011 .
 - **IEEE 32-bit:** $0\ 10000101\ 11110000101000111101011$

Q9. Multiply following operands using recoded multiplier technique: $(+12 \times -3)$, $(+16 \times -7)$

The recoded multiplier technique (Booth's recoding / Bit-Pair Recoding) groups multiplier bits into pairs/triplets to reduce the number of partial product addition steps.

Part (a): $+12 \times -3$

- Multiplicand $M = +12$ (01100)
- Multiplier $Q = -3$ in 5-bit 2's complement: 11101 (with padded $Q_{-1} = 0 \rightarrow 111010$).
- **Recoding Pairs:**
 - 10 $\rightarrow$ subtract M
 - 01 $\rightarrow$ add M
 - 00 or 11 $\rightarrow$ add 0
- Executing recoded additions yields product -36 (1111101100 in 2's complement).

Part (b): $+16 \times -7$

- Multiplicand $M = +16$ (10000)
- Multiplier $Q = -7$ (11001)
- Recoding and summing partial products gives -112.
- **Final Product:** -112.

Q10. Multiply the following pair of signed 2's complement numbers using bit-pair-recoding Booth's Algorithm: Multiplicand: 110001, Multiplier: 101101

Given: Multiplicand $M = 110001$ (signed negative, value = -15), Multiplier $Q = 101101$ (signed negative, value = -19).

Bit-Pair Recoding (Modified Booth's Algorithm) Steps:

1. Extend multiplier with $Q_{-1} = 0$ to form groups of 3 bits: (y_{i+1}, y_i, y_{i-1}) .
2. Groups for 101101 0:
 - Group 1 (rightmost): (1, 0, 0) $\rightarrow$ Recoded value: $-2M$
 - Group 2: (1, 1, 0) $\rightarrow$ Recoded value: $-1M$
 - Group 3: (1, 0, 1) $\rightarrow$ Recoded value: $0M$
3. Shift and accumulate partial products according to recoded radix-4 digits.
4. **Result:** Multiplying $-15 \times -19 = +285$. In binary 2's complement representation: 00000100011101.

Q11. Perform division of following numbers using Restoring Division Algorithm: Dividend = 1000, Divisor = 0001; and Dividend = 23, Divisor = 5

Part (a): Dividend = 1000 (8), Divisor = 0001 (1)

- **Initialization:** $A = 0000$, $Q = 1000$, $M = 0001$, $n = 4$.
- **Cycle 1:** Shift left AQ. $A = 0001$, $Q = 0000$. Compute $A = A - M = 0001 - 0001 = 0000$. Since $A \geq 0$, set $q_0 = 1$.
- **Cycle 2-4:** Repeat left shift and subtraction. Since divisor is 1, quotient bits accumulate correctly.
- **Final Quotient:** $Q = 1000$ (8), Remainder $A = 0000$ (0).

Part (b): Dividend = 23 (1011), Divisor = 5 (00101)

- **Initialization:** $A = 00000$, $Q = 1011$, $M = 00101$, $n = 5$.
- **Step-by-step Execution:**
 - Shift left AQ iteratively, subtract M. If remainder < 0 , restore A ($A = A + M$) and set $q_0 = 0$; else set $q_0 = 1$.
- **Final Quotient:** 00100 (4 in decimal).
- **Final Remainder:** 00011 (3 in decimal). ($23 = 5 \times 4 + 3$).

Q12. Perform division of following numbers using Non-Restoring Division Algorithm: Dividend = 1010, Divisor = 0011; and Dividend = 15, Divisor = 3

Part (a): Dividend = 1010 (10), Divisor = 0011 (3)

- **Initialization:** $A = 0000$, $Q = 1010$, $M = 0011$, $n = 4$.
- **Algorithm Steps (Non-Restoring):**
 1. If sign of A is 0, shift left AQ and compute $A = A - M$. Else shift left AQ and compute $A = A + M$.
 2. If final $A \geq 0$ ($msb = 0$), set $q_0 = 1$. Else set $q_0 = 0$.
 3. Repeat for n cycles. If final remainder $A < 0$, perform a final restoring addition $A = A + M$.
- **Execution Results:** Quotient $Q = 0011$ (3), Remainder $A = 0001$ (1). ($10 = 3 \times 3 + 1$).

Part (b): Dividend = 15 (0111), Divisor = 3 (00011)

- **Initialization:** 5-bit representation. $A = 00000$, $Q = 0111$, $M = 00011$, $n = 5$.
- **Execution:** Non-restoring division tracks running sign bits, subtracting or adding M accordingly.
- **Final Quotient:** 00101 (5 in decimal).
- **Final Remainder:** 00000 (0 in decimal). ($15 = 3 \times 5 + 0$).